Generative UI with OpenUI: From Zero to Hero

How AI-built interfaces actually work - the complete mental model, told with a restaurant, a menu, and a chef who never enters your kitchen.

Part I: the mental model in ten stops | Part II: delivery considerations | October 2026

Chatbots answer with walls of text. Apps show screens you can act on. **OpenUI** (open source, by Thesys) is a framework for a third way - **Generative UI** - where the AI builds the interface itself, live, tailored to each question. This article distills an interactive course into one piece: the concepts, the vocabulary, and the mental model you need before writing a single line of code. **Part I** builds that mental model in ten stops. **Part II** answers the questions that decide a real delivery: your own chat UI, the built-in component catalog, self-hosting, theming, authentication, and plain HTML.

How to use the index: every entry below is clickable, and the same list lives in the PDF bookmarks sidebar of your reader.

Index

PART I - THE MENTAL MODEL

1. Why OpenUI Exists - from walls of text to living screens

2. The Big Idea - a restaurant with a remote chef

3. The Core Loop - journey of a message

4. OpenUI Lang - reading a recipe

5. Component Libraries - the menu and its two faces

6. The Toolbox - who does what

7. Who Runs the AI? - Gateway vs. self-hosted

8. The Chat Shell - the dining room

9. Making It Alive - buttons, data and artifacts

10. Hero Mode - breaking, fixing, shipping

The Complete Mental Model - one page

PART II - DELIVERY CONSIDERATIONS

11. Using Your Own Chat UI - react-headless and the three levels

12. The Built-in Component Catalog - what is on the menu

13. Going Open-Source - what changes when you self-host

14. Custom Theming - repaint the palette, not the furniture

15. Authentication - your app is the front door

16. Plain HTML - the browser bundle and generated HTML

1 · Why OpenUI Exists

You have used ChatGPT-style assistants. You ask something, and you get **a wall of text**: "Here are 3 flight options. Option A departs at 9am, costs $420, has 1 stop..." Now compare a real travel app: cards, prices, a Book button you can actually press.

Text describes. Screens let you act.

The two old options - and why both hurt. Build every screen by hand, and a team spends weeks producing interfaces that are frozen in time, unable to adapt to what each user wants. Or let the AI just talk: fast to build, but the user does all the work in their head.

The third option: Generative UI (GenUI). The AI creates the screen itself, on the spot. Ask "show me flights under $400" and it builds flight cards with working buttons. Ask "how were my sales?" and it draws a real chart instead of a paragraph of numbers.

The scary question - and the trick. If the AI writes the interface, can't it write broken or dangerous code? This is the problem OpenUI solves, and its answer is the heart of everything that follows: the AI is **not allowed to write code**. It writes instructions in a tiny, restricted language, using only components you approved in advance - like a chef who can only cook dishes from a fixed menu. Creative within limits. Incapable of chaos.

Who is this for? Anyone building assistants that should do things, not just talk: a banking assistant showing your actual transactions, a data assistant drawing charts on demand, a shopping assistant with product cards, a support agent displaying your real order status.

Requirements to follow along: zero. No coding, no setup, no prior AI knowledge. If you know what a chatbot and a browser are, you are qualified.

> **Key takeaways**
>
> Chatbots describe; GenUI lets users act.
>
> Hand-built screens are slow and frozen; text-only assistants are weak.
>
> OpenUI = a complete framework for AI-built interfaces, safely.
>
> Safety trick: a restricted recipe language plus a pre-approved menu of components.

2 · The Big Idea: A Restaurant With a Remote Chef

One analogy. Everything in OpenUI maps to it. Imagine a restaurant where the chef is brilliant but **works from another country**. He cannot enter your kitchen - he can only **send recipes**.

In the restaurant	In OpenUI
You, the customer	The user typing in the chat
Your order at the counter	Your message
The remote chef	The AI model (Claude, GPT, ...)
Your kitchen	The app running in the browser
The menu	The component library (Stop 5)
The recipe format	OpenUI Lang (Stop 4)
Kitchen staff who cook	The renderer (Stop 3)

Rule 1 - the chef can only cook from the menu. The AI cannot invent UI out of thin air. It gets a list of approved building blocks - cards, charts, tables, buttons, forms - and must compose from those. That is the safety guarantee.

Rule 2 - recipes follow a fixed, simple format. Not freeform code: a tiny language where each line is one instruction. Simple enough that the AI rarely gets it wrong, strict enough that mistakes are easily caught.

Rule 3 - your kitchen staff does the actual cooking. Recipes are just instructions. The browser-side **renderer** assembles the real, interactive interface. The AI never touches your kitchen - it only sends paper.

Why this split is genius. Because chef and kitchen are decoupled by paper:

- **Safe** - a recipe cannot contain a virus any more than a recipe card can start a fire.
- **Swappable** - change chefs (GPT to Claude to whatever is next) without touching the kitchen; the recipe format is the contract.
- **Customizable** - add your own dishes to the menu (custom components), teach the chef once, and he cooks them forever.
- **Streamable** - recipes arrive line by line, and the kitchen starts plating the moment the first line lands. Users watch the UI build itself.

> **The one sentence to hold onto:** the AI writes recipes from a fixed menu; your browser cooks them.

3 · The Core Loop: Journey of a Message

Let's follow one real message through the restaurant. You type: "How were my sales this quarter?" - plain text in the chat box, and the journey begins.

1 - The server prepares the briefing pack. The app's server cannot just forward your raw question. It packs an envelope: the **menu** (every component the AI may use), the **recipe grammar**, **house instructions** ("prefer charts for trends, keep tables short"), and **your question**. This envelope is the **system prompt** - the single most important document in the system: the AI's complete job description.

2 - The chef writes the recipe in the tiny language: a Stack containing a title, two StatCards, a table.

3 - Streaming: the chef dictates, does not mail. The recipe streams back **line by line**, like a fax printing while the chef speaks. A full recipe takes seconds; nobody wants to stare at a spinner that long.

4 - The parser reads the fax. In your browser, the **parser** reads each line as it lands and builds a structured plan. A malformed line is noted as an error instead of crashing.

5 - The renderer cooks - immediately. The moment the first line (the layout frame) arrives, something appears on screen; as more lines land, details fill in. **This is why you watch the UI assemble itself live.**

6 - Done: a real, live interface. Clickable, hoverable, scrollable. Not a screenshot - actual UI, built seconds ago, personalized to your question.

7 - You act, the loop repeats. A click becomes a new message; back to step 1. Conversation and interface feed each other.

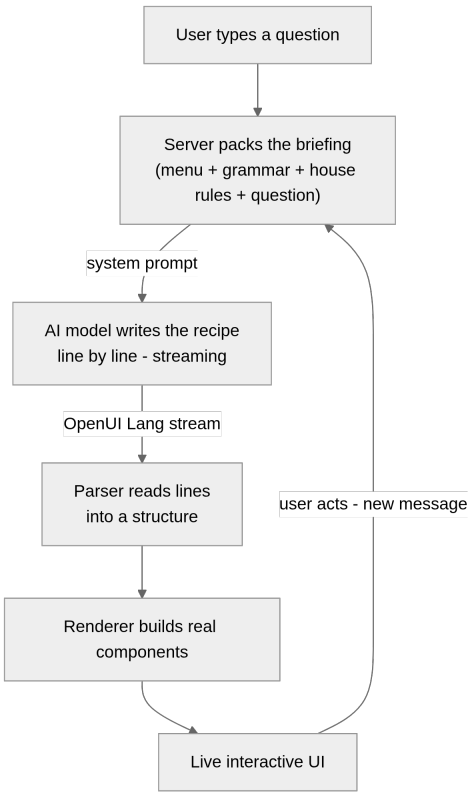


Diagram 1 - The whole journey at a glance: briefing out, recipe stream back, live UI, and the action loop.

The four crew members, gathered in one place:

Crew member	Job	Name
Briefing pack	Tells the AI what it may use and how	System prompt
Chef	Writes the recipe	The AI model
Reader	Turns recipe lines into a structured plan	The parser
Cook	Builds the real UI from the plan	The renderer

4 · OpenUI Lang: Reading a Recipe

This is not "learning to code" - it is learning to **read an instruction sheet**, like IKEA directions. Here is the entire interface for "How were my sales this quarter?", five lines:

```
root = Stack([title, cards])
title = TextContent("Q4 Sales Dashboard", "large")
cards = Stack([revenue, orders], "row")
revenue = StatCard("Revenue", "$1.2M")
orders = StatCard("Orders", "8,214")
```

Line 1: the stage is a **Stack** (a vertical container) holding two things, title and cards. **root is magic** - it is always the outermost thing on screen, and the kitchen puts it up first.

Line 2: where title goes, place a TextContent - those words, sized large. **Line 3:** another Stack holding revenue and orders, arranged in a row. **Lines 4-5:** two labeled number tiles. A dashboard, built from four menu items.

The whole language is one pattern: **name = Component(details, ...)** - "make a thing, call it this name, here are its details." Five rules govern everything:

#	Rule	Plain meaning
1	root must exist	Something must be the stage - otherwise nothing renders
2	root comes first	The frame goes up before the decorations (streaming)
3	One instruction per line	A checklist, not an essay
4	[brackets] mean "place that named thing here"	Containers point to other lines
5	Details go in a fixed order	Like form fields: skip from the end, never shuffle

The cleverest trick: line 1 references title and cards before they are defined. That is allowed - and deliberate. The chef announces the plan first, then fills in the cast, like a playbill printed before any actor is introduced. Mid-stream, the kitchen already has the frame on screen and simply hangs each new piece as its line arrives.

Why this language instead of normal code? It is designed for a **machine writer and a machine checker**: no logic, no loops, nothing to remember; a misspelled component name is instantly detectable; and the AI picks which components and what data, but can never invent new building blocks or sneak anything past the menu.

5 · Component Libraries: The Menu and Its Two Faces

The chef can only cook what is on the menu. So who writes the menu - and what is actually on it? That is the **component library**, the closest thing OpenUI has to a soul.

Every dish on the menu has **two faces**:

Face	Who reads it	What it says
The description	The chef (AI), via the briefing pack	"FlightCard - one flight option: airline, times, price, with a Book button."
The implementation	The kitchen (browser)	The actual visual component that draws a real, clickable flight card

> **The rule the entire framework stands on:** the two faces must always match. If the menu promises a FlightCard the kitchen cannot build, the user sees a blank space or an "unknown component" error.

Each entry also declares **its fields, in order** - "airline first, then times, then price" - exactly what the chef follows when writing FlightCard lines. This description-plus-implementation pair is **the contract**; keeping it in sync is most of real-world GenUI debugging.

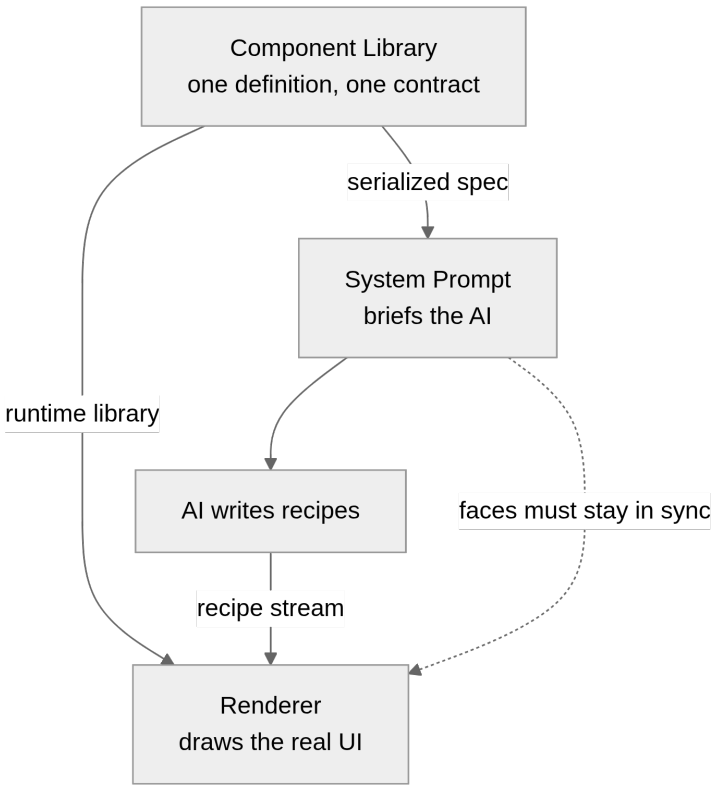


Diagram 2 - One library definition, two projections: the prompt briefs the chef, the runtime draws the UI.

Two ready-made menus. OpenUI ships two built-in libraries: **openuiLibrary** (the full general-purpose set: charts, tables, forms, cards, layout, modals - for apps) and **openuiChatLibrary** (a curated chat-friendly subset with follow-up chips, step lists and callouts - tuned for assistant replies). For many apps you just pick one and you are done.

Writing your own dishes. Your app has domain things no generic menu covers - a TransactionRow for banking, a FlightCard for travel, a WorkoutSummary for fitness. Define it once (description for the chef, implementation for the kitchen, fields in order), and the AI cooks it like any built-in. The AI never saw your design system; it just read your menu.

Menu design is psychology for chefs. The AI chooses components by reading descriptions, like a customer ordering from how the waiter describes dishes:

- **Clear, distinct names win.** Two dishes that sound alike means wrong orders - overlapping components are a top source of flaky output.
- **Smaller menus are better.** Every extra dish lengthens the briefing and adds a way to hesitate. Do not expose your entire design system.
- **Field order is communication.** Important fields first; the chef leads with them.

> A beautiful app with a badly described menu generates badly. A simple app with a crisp menu generates reliably. Menu quality is output quality.

6 · The Toolbox: Who Does What

OpenUI ships as a set of packages. Here is the cast list, translated into plain words:

Package	Plain words	Restaurant role
@openuidev/lang-core	The recipe language itself: grammar, reader (parser), briefing writer. Knows nothing about any kitchen brand.	Head-office manual
@openuidev/react-lang (also vue-lang, svelte-lang)	Kitchen staff trained for one kitchen brand: define menu entries, cook recipes.	Trained cooks
@openuidev/react-ui	The full React package: built-in menus, furniture, theming - and AgentInterface, the dining-room-in-a-box.	Franchise interior kit
@openuidev/react-headless	All the kitchen equipment, zero furniture - build your own dining room.	Invisible machinery
@openuidev/cli	Launch kit: create a restaurant from a template, generate the printed menu, deploy for business.	Franchise starter box
@openuidev/devtools	A kitchen camera - inspect every incoming recipe, replay failures frame by frame.	Debug feed
@openuidev/observability-cloud	The health inspector for live service - reports broken dishes in production.	Inspector

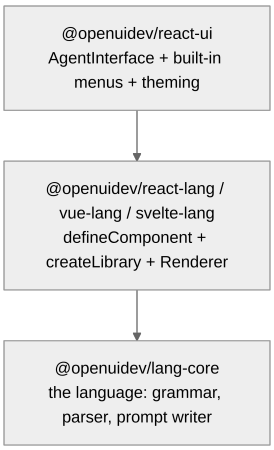


Diagram 3 - The stack: each floor stands only on the one below.

Two things fall out of this stack. First, **the language does not care where it is cooked**: lang-core is kitchen-brand-neutral, so the same recipe format works in React, Vue or Svelte - only the cooks differ. Second, **most people only ever touch two boxes**: react-ui (furniture, menus, dining room) and the cli (create / generate / deploy). The rest exists for special occasions.

7 · Who Runs the AI? Gateway vs. Self-Hosted

The chef is remote - but **who employs him?** This is the one big decision in every OpenUI app, and there are exactly two answers.

Option A: the staffing agency - OpenUI Gateway. Thesys runs a hosted service between your app and the AI models. Your app never talks to Claude or GPT directly - it talks to the agency, which dispatches a chef. What you get beyond the obvious:

- **Model routing with fallbacks** - if one chef is unavailable, the agency sends a substitute; you wrote the request once.
- **A recipe proofreader** - an editor fixes small errors in-flight (a typo'd component name, a malformed line) before delivery. Plain-text chat gets no such treatment.
- **Hosted extras** - web search, image search: chefs who can pop out to the library mid-recipe.
- **Optional thread storage** - the agency can keep conversation history in its filing cabinet, so users reopen old chats. Optional, not automatic.

The deal: one contract, one bill, no per-chef paperwork. You give up some control - the agency picks the substitute.

Option B: contract the chef yourself - self-hosted / direct. Your app calls the AI provider with your own API keys. You choose the exact chef and own everything: reliability, retries, cost. No proofreader, no substitute. Choose this when control is the requirement - data residency, a mandated model, your own accounts.

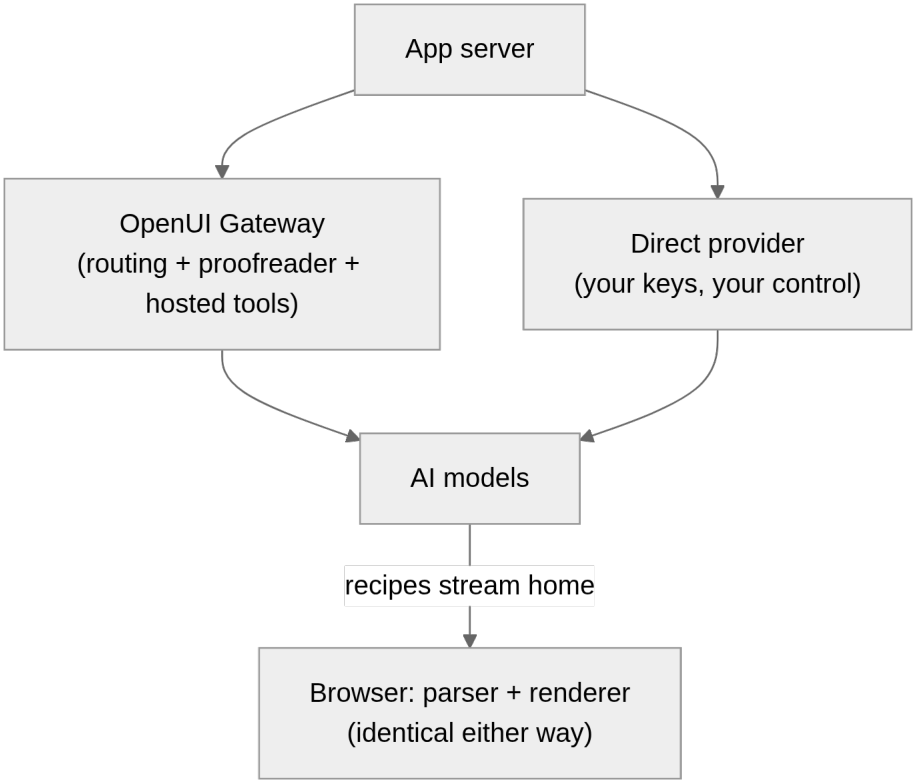


Diagram 4 - Two ways to employ the chef; the browser side is identical either way.

> **The plot twist:** both options speak the identical recipe language, and the browser side is byte-for-byte the same. Swapping later is a server-side errand, not a rebuild.

One non-negotiable security rule. The contract stays in the office: the API key (yours or the agency's - THESYS_API_KEY) lives **only on the server**, never in browser code. If the browser needs saved threads, it gets a **limited visitor badge** (a scoped frontend token) that opens only that user's drawer.

And one mindset rule. The choices are **independent dials**, not one bundle: who generates, who stores history, which protocol, what the chat looks like, which extras are on. Picking one dial does not twist the others. Beginners fuse these; you will not.

8 · The Chat Shell: The Dining Room

A restaurant is more than a kitchen. There is a **dining room** - and in OpenUI it is called **AgentInterface**: a complete, prebuilt chat app you drop into your project.

What is in the box: the conversation view (messages flowing, recipes rendering live), a sidebar listing past conversations you can reopen, the composer with suggested starter prompts, a welcome screen, branding dials (name, logo, colors, light/dark), and a responsive layout for phones. Use it as-is, restyle it, swap pieces - or build your own with the equipment-only kit. The kitchen does not care who built the dining room.

A chat shell needs exactly **two pipes** plugged in - and they are **independent**:

Pipe	Job	Required?
The waiter (llm)	Carries your order out, brings recipes back. Connects to a route on your server.	Yes - no waiter, no food
The reservations book (storage)	Keeps conversations so users can come back to them.	No - without it, chats live in memory

Who keeps the book is your choice: nobody (everything forgotten on refresh - fine for demos), the agency's cabinet (threads in Gateway cloud storage), or **your own cabinet** (your database, your rules). Pair any waiter with any book - all combinations work.

The visitor badge, revisited. If threads live in the agency's cabinet, the browser cannot hold the master key. Your server mints each logged-in user a scoped badge that can open only their own drawer of the filing cabinet. The shell carries the badge for you.

Peek at the floor plan and there is one more room: a **workspace panel** off the dining area - built for artifacts, the headline act of the next stop.

9 · Making It Alive: Buttons, Data and Artifacts

So far our screens are beautiful but passive - a magazine photo of a restaurant. Time to wire up the electricity. Five upgrades, each small, each huge.

1. The notepad - reactive state. Some interfaces need a memory: pick "7 days" in a dropdown and a total updates - without bothering the chef. A recipe line like `$days = "7"` creates a variable. Input fields write to it; text on screen reads from it; both update instantly. A live, working form in two recipe lines.

2. Verbs - actions. Buttons need to do things. Special verbs, always written with an @ so they are unmistakable: `@Set` / `@Reset` (write to / clear the notepad), `@OpenUrl` (open a link), `@Run` (trigger something bigger). A "Book now" button is: on click, @Run(bookFlight). Which raises the question - run what, exactly?

3. The service elevator - live data. The kitchen is in the browser, but real data lives on servers. Recipes can use the elevator:

- **Query** - "fetch this when the screen opens, and refetch when a notepad value it depends on changes." Change 7 days to 30 and the table refreshes by itself.
- **Mutation** - "perform this action on the server when triggered." That @Run(bookFlight) runs a Mutation: your server books the flight, the screen updates.

Crucial: the elevator leads to **your** building. The recipe only names the endpoint; your app executes it - your rules, your permissions, your validation. The chef names the dish; your staff handles the food.

4. Errands - tools. Some recipes need things only the restaurant can arrange: "search the web", "look up this user's orders". The chef can request errands - but with a hard rule: **the chef may request an errand; only the app may run it.** Errands are declared in the briefing; when one appears in a recipe, your server decides whether to allow it and executes it with full authentication. A requested-but-undeclared errand simply does not run. The chef suggests; the app disposes.

5. The display case - artifacts. Some creations are too big for a chat bubble - a full report, a dashboard, a presentation. The flow: the AI calls an app tool ("file this report"), **your app** produces the content, a renderer **your app provides** displays it - inline as a preview card and full-screen in the workspace panel - and it is saved for later: users reopen it days later, browse old artifacts in the sidebar, edit if you allow it. Chat is the conversation; **artifacts are the products**.

Rung	Power	Needs the chef?
Static UI	Rendered views	Once, to create
Notepad (state)	Forms that remember	No
Actions	Buttons that react	No

Rung	Power	Needs the chef?
Query / Mutation	Live server data	No - elevator to your backend
Tools	Web search, lookups	Requests it; the app executes
Artifacts	Browsable creations	Requests it; the app produces

10 · Hero Mode: Breaking, Fixing, Shipping

An AI is not a compiler. Same question, ten runs: nine perfect recipes - and one where the chef **invents a dish that is not on the menu**, puts details in the wrong field order, mentions chart but never defines it, or **stops writing mid-recipe** - frame on screen, empty inside. This is normal. Nondeterminism is not a bug in OpenUI; it is the nature of AI, and the entire architecture is a chain of nets catching it.

Layer	What it catches
Constrained language	Few ways to go wrong in the first place
Disciplined menu	Clear names, small set - the chef rarely hesitates
Careful briefing	Rules and examples for tricky shapes
Gateway proofreader	Fixes eligible errors in-flight, before you see them
Correction loop	Parser errors fed back: "chef, line 3 is misspelled" - rewritten
DevTools (development)	Inspect any stream, replay failures without re-calling the chef
Observability (production)	Reports which live dishes came out broken

> **The hero's golden rule:** never trust one success. Test real prompts repeatedly; count structural errors and partial renders, not just blank screens; re-measure after every menu or briefing change.

Notice the pattern: **prevention lives in your design choices** (menu, briefing), **cure lives in the pipeline** (proofreader, correction loop), **visibility lives in tooling** (DevTools in dev, Observability in production). Heroes spend their energy on prevention - the cheapest layer.

Shipping it. A restaurant on your laptop is not a restaurant. When the app works locally, one command packages the whole thing - kitchen, dining room, menus - and opens it on a **public preview URL** you can send to anyone; the same command with --prod opens for real users (that is the cli doing create, then deploy). Localhost is rehearsal; deploy is opening night.

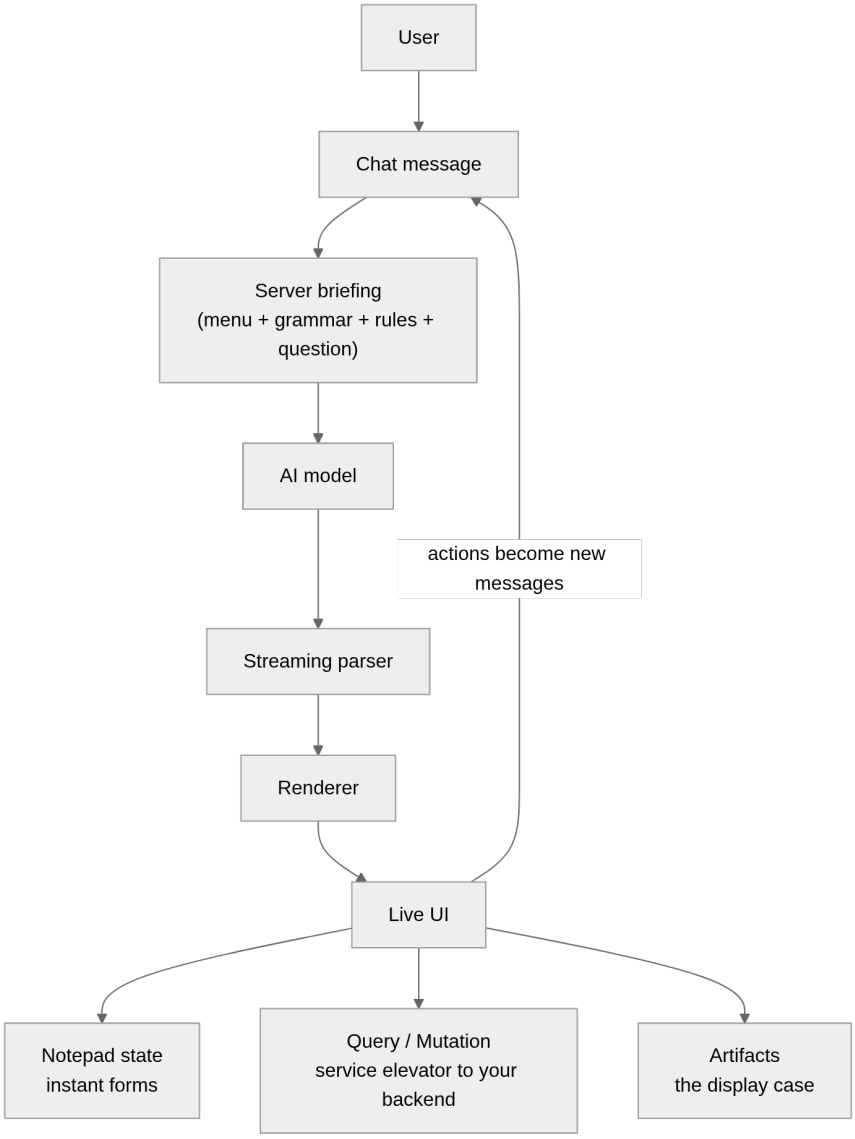


Diagram 5 - The complete mental model: one loop, three side systems, endlessly repeated.

The Complete Mental Model - One Page

> **The one sentence:** the AI writes recipes from a fixed menu; your browser cooks them.

Memory hook	The rule
The contract	Every menu item has two matching faces - description for the chef, implementation for the kitchen. Keep them in sync: 90% of real debugging.
The five independent dials	Who generates (Gateway / direct), who stores history, which protocol, what the shell looks like, which extras are on. Twist each separately.
The security trinity	Keys stay on the server. Browsers get scoped badges. The app executes everything sensitive - the AI only requests.
The loop	Briefing out, recipe stream back, parser reads, renderer cooks, live UI - and every action becomes a new message.
Reliability	Prevention in design (menu, briefing), cure in the pipeline, visibility in tooling. Never trust one success.

PART II - DELIVERY CONSIDERATIONS

11 · Using Your Own Chat UI

Stop 8 gave you the dining room in a box. But what if you already have a dining room - your own chat interface, your own design system? Two build-floor questions answer this: what exactly is **react-headless**, and how do you plug the kitchen into **your** UI?

What react-headless is. It is the equipment-only package: chat state and hooks, stream adapters, message-format converters, and storage interfaces - with **zero visual components**. No sidebar, no composer, no styling. "Headless" means all the brain, no face: you get everything that makes chat work and paint the face yourself. One detail saves confusion: react-ui **re-exports** everything in react-headless, so the machinery is available either way - import from react-headless when you do not want the visual package in your bundle at all.

Three levels of "own UI", lightest first:

Level	Your situation	The move
1	A chat UI already exists (your own, assistant-ui, CopilotKit)	Keep the shell; render recipes inside your message bubble with the Renderer
2	You are building a new chat with your own design	react-headless owns state and streaming; you draw every pixel
3	Your backend speaks an unusual transport	Implement the ChatLLM interface: your own send() plus a stream adapter

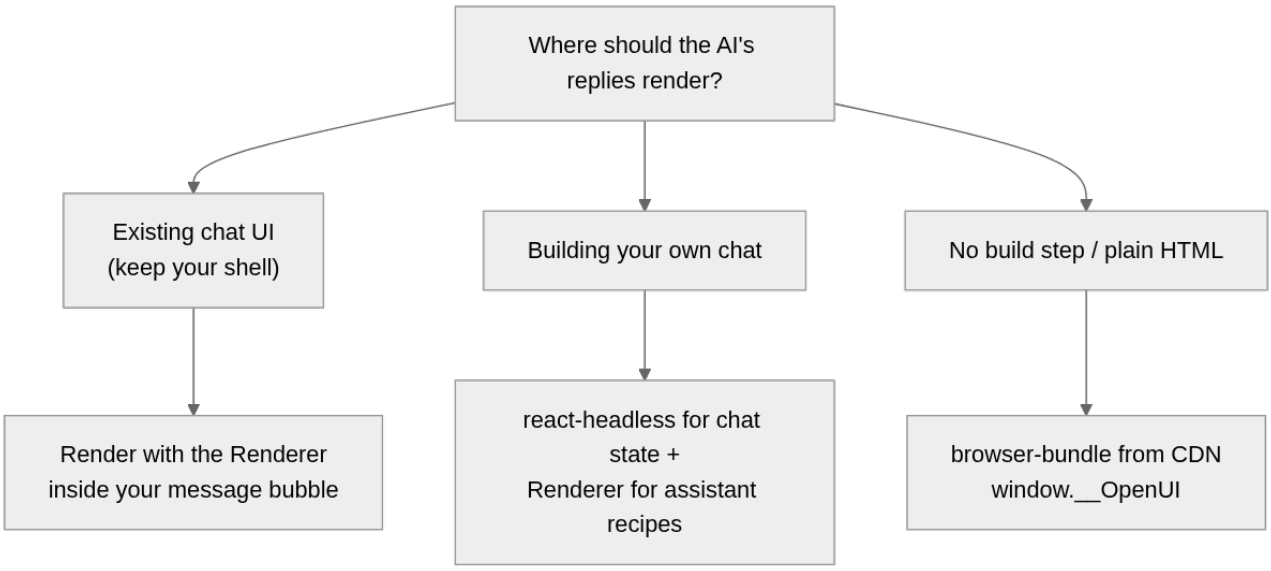


Diagram 6 - Choosing the integration level for your own chat UI.

Level 1 - the lightest touch. Your app keeps its own message state and styling. The only change: an assistant message is now a recipe, so the bubble renders it with the cook:

```
import { Renderer } from "@openuidev/react-lang";
import { openuiChatLibrary } from "@openuidev/react-ui";
import "@openuidev/react-ui/styles/index.css";  // once, for styles

function AssistantMessage({ response, isStreaming }) {
  return (
    <Renderer response={response} library={openuiChatLibrary}
              isStreaming={isStreaming} onError={console.error} />
  );
}
```

Level 2 - your chat, OpenUI's machinery. Wire the waiter yourself and feed it to the headless chat state; render the message list with your own components and each assistant recipe as in Level 1:

```
import { fetchLLM, openAIAdapter, openAIMessageFormat }
  from "@openuidev/react-headless";

const llm = fetchLLM({
  url: "/api/chat",                 // your backend route
  streamAdapter: openAIAdapter(),   // must match what the browser receives
  messageFormat: openAIMessageFormat,
});
```

What never changes, at any level:

- The contract - the server still briefs the chef with generateSystemPrompt and the same library the Renderer uses; mismatch still means blank dishes.
- The adapter matches the browser's actual bytes, not what the server calls internally.
- The two pipes stay independent - your UI can still use cloud storage, your own storage, or none.

> One-line decision guide: already have a chat UI, Level 1; building a new one with your own design, Level 2; unusual transport, Level 3.

12 · The Built-in Component Catalog

What is actually on the menu? This inventory is verified against the open-source repository itself (the genui-lib folder of @openuidev/react-ui: 59 components) plus the live docs. Remember: the menu is a browser-side artifact, so this catalog is identical on the Gateway and self-hosted paths.

Layout and containers

Component	What it does
Stack	The core container - stacks children vertically/horizontally (library root)
Card / CardHeader	Styled card container / its title-subtitle header
Tabs + TabItem	Tabbed panels (value + trigger + content per tab)
Carousel	Horizontally swipeable slides
Accordion	Expand/collapse sections
Modal	Dialog overlay
Separator / InlineHeader	Visual divider / inline section heading

Text and content

Component	What it does
TextContent / Text / BoldText	Text at a chosen size / plain / emphasized
Label / IconText / Tag	Caption / icon-plus-text pair / small pill badge
Icon / IconButton	Icon glyph / clickable icon
CodeBlock	Syntax-styled code snippet
MarkDownRenderer	Renders markdown content

Charts (nine types)

Component	What it does
BarChart / HorizontalBarChart	Vertical bars / side-lying bars for long labels
LineChart / AreaChart	Trends over a series / filled trend
PieChart / SingleStackedBarChart	Part-to-whole slices / one bar, stacked segments
RadarChart / RadialChart	Multi-axis comparison / circular gauge
ScatterChart	Point clouds and correlations

Tables and forms

Component	What it does
Table + Col	Data table defined by columns: label + data array + type (e.g. currency)
EditableTable	Table whose cells the user can edit
Form + FormControl	Named form bundling fields and buttons / labeled field wrapper
Input / TextArea / Select	Text field / multi-line / dropdown
DatePicker / Slider	Date chooser / numeric range
CheckBoxGroup / RadioGroup / SwitchGroup	Multi-select / single-select / toggles
OptionCards	Choose-one card options

Buttons, actions and images

Component	What it does
Button / Buttons	Labeled button wired to an action / grouped buttons
Action	The action wrapper (@Run, @Set, @OpenUrl...)
Image / ImageGallery	A single image / browsable collection

The chat library (root: Card, no Stack)

Component	What it does
Callout / TextCallout	Highlighted note or warning boxes
Steps	Numbered step-by-step list
ListBlock + ListItem / SectionBlock + SectionItem	List / sectioned content blocks
FollowUpBlock + FollowUpItem	Suggested next-question chips
EntityList / Chips	Named-entity list / selectable tags
MetricIndicator	Compact metric/KPI display
Card blocks: Overview, Snippet, Composite, Context, Visual, Image, ImageText, ImageTextLarge, TagBlock	Media-and-text composition blocks for Card
Skeleton / TableSkeleton	Loading placeholders while data streams
Query / Mutation (non-visual)	The service-elevator data fetchers - executed by your backend

Two honest notes

Membership differs per library: openuiChatLibrary is a curated Card-rooted subset (no Stack); openuiLibrary is the full suite. The version-exact catalog is always the generated prompt - openuiLibrary.prompt() in code, or the CLI's generate command.

Course correction: our Stop 4 example used StatCard, which is illustrative. The current source shows the metric component as MetricIndicator - always verify names against your installed version.

13 · Going Open-Source: What Changes

Choosing Option B from Stop 7 - self-hosting, your own model keys - is a service-staff decision, not a menu decision. The punchline: the component catalog above is a browser-side artifact, so it is **identical** on both paths. What changes is everything around the chef:

Concern	Gateway path	Your open-source path
Components, parser, renderer	Same	Same - the catalog above
System prompt	generateSystemPrompt with cloud: true	Same helper, no cloud flag
Recipe proofreader	Built in - fixes eligible errors in-flight	Gone - you build the correction loop: catch parser errors, feed them back to the model
Model access	One THESYS_API_KEY	Your own provider keys, server-side only
Hosted tools (web/image search)	Provided	You wire your own tool executions
Thread storage	useOpenuiCloudStorage	restStorage or your own ChatStorage

The practical consequence: **reliability becomes your job**. Stop 10's nets do not disappear - the constrained language and disciplined menu still prevent, and DevTools still expose - but the in-flight proofreader is gone, so your correction loop and prompt discipline carry the weight.

The one verification habit: once the package is installed, regenerate the authoritative catalog from your installed version (the CLI's generate command against your library file). If any name differs from this article, trust your installed version - that is the contract your chef actually sees.

14 · Custom Theming

Repaint the palette, not the furniture. Every built-in component reads its colors from shared design tokens (CSS variables named --openui-*). You do not restyle components one by one - you replace the palette, and every card, button, chart and chat bubble repaints itself. Three moves:

Move 1 - define your brand as partial theme objects, once at module level, mapping your app's own CSS variables. Anything you skip falls back to built-in defaults:

```
import { createTheme } from "@openuidev/react-ui";

export const brandLightTheme = createTheme({
  background: "var(--app-canvas)",
  interactiveAccentDefault: "var(--app-accent)",
  interactiveAccentHover: "var(--app-accent-hover)",
  chatUserResponseBg: "var(--app-user-message)",
  radiusM: "10px",
});
```

Move 2 - choose exactly one theme owner:

Situation	Pattern
Only the chat needs theming	Pass the envelope to AgentInterface.theme
A broader app tree uses OpenUI components	Wrap that tree in ThemeProvider
ThemeProvider already wraps AgentInterface	Add disableThemeProvider on AgentInterface - never both providers by accident

Move 3 - light/dark is your app's decision. The provider owns no toggle, no persistence, and accepts no "system" mode: your app resolves the preference and passes "light" or "dark":

```
<AgentInterface llm={llm}
  theme={{ mode, lightTheme: brandLightTheme, darkTheme: brandDarkTheme }} />
```

Pitfalls worth burning in: theme takes the envelope (mode/lightTheme/darkTheme), not the bare token object; do not wrap in an outer ThemeProvider while the interface's internal provider is still on - the inner one silently resets your brand tokens; keep importing the CSS once (use the layered variant if you need cascade-layer overrides, never both); changing font primitives does not regenerate compound typography tokens; and token names are version-specific - verify against your installed release.

Verify the result

Toggle modes and confirm the computed --openui-* values change without console warnings.

Check contrast in both modes: buttons in every state, inputs, user messages, dropdowns, modals.

Inspect the injected style tag (data-openui-theme) matches the intended scope.

15 · Authentication

OpenUI ships no front door - you do. On the self-hosted path, your existing login (session cookie, JWT - whatever your app already uses) is the authentication system. The chat renders inside pages your app already protects. But there is one trap to burn in:

> **Page auth is not API route auth.** Protecting the page does not protect the endpoint. Your /api/chat route must verify the session on every request - or anyone can call it directly and spend your model tokens.

The four locks to install:

#	Lock	What it means
1	Generation route checks identity	/api/chat reads the session server-side on every request; rejects logged-out callers; rate-limits; validates body shape and size; accepts model ids only from your server-side allowlist
2	Provider keys stay server-side	Your OpenAI/Anthropic keys live in server env only - never in browser code or bundles
3	Storage is scoped by user	Every thread read/write filters by the authenticated user id; restStorage has no built-in auth - put it behind your own checked endpoints
4	Replace demo identity	Templates ship a DEMO_USER_ID for local play - swap in real session identity before production

The tests that prove it works:

- **Logged-out request rejected** - hit /api/chat with no session: expect 401/403.
- **Cross-user isolation** - as user A, request a conversation id belonging to user B: rejected. The app must check ownership, not just login.
- **No keys in the browser** - search the client bundle for provider key names: zero hits.

Nail those three plus rate-limiting, and your self-hosted chat is as secure as the rest of your app - because it is the rest of your app.

16 · Plain HTML

Yes - the browser bundle. @openuidev/browser-bundle is the "no kitchen installation required" kit: the entire renderer - React, the parser, the component library - pre-packed into one JS file plus one CSS file, loaded from a CDN. No build step, no npm, no framework:

```
<link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/@openuidev/browser-bundle@latest/dist/openui-styles.css" />
<script src="https://cdn.jsdelivr.net/npm/
  @openuidev/browser-bundle@latest/dist/openui-bundle.min.js"></script>

<div id="root"></div>
<script>
  const { React, createRoot, Renderer, openuiChatLibrary } = window.__OpenUI;
  const recipe = "root = Card([ ... ])";  // OpenUI Lang from your backend
  createRoot(document.getElementById("root")).render(
    React.createElement(Renderer, {
      code: recipe,
      library: openuiChatLibrary.library,
      schema: openuiChatLibrary.schema,
    })
  );
</script>
```

The bundle attaches a single global, **window.__OpenUI**, exposing exactly four things - React, createRoot, Renderer, openuiChatLibrary - and that shape is a declared **stable public contract** (changes require a major version bump).

Question	Answer
How big?	About 650 KB gzipped JS plus 150 KB CSS - the price of "no build"
Where from?	jsDelivr or unpkg; pin a version instead of @latest for production
When not to use it?	Any project with a build pipeline - use react-lang and react-ui directly (smaller, typed)
Official use cases	Embeddable widgets on static sites, iframe/sandboxed plugin responses, prototypes

Two things do not change: the **contract** (your server briefs the chef with the same library spec, or dishes come out blank) and the **loop** (sending the user's message, streaming the recipe back, and updating the code as chunks arrive is still your plumbing - a plain fetch and re-render).

The sibling pattern: model-generated HTML. If the requirement is for the AI to output raw HTML instead of OpenUI Lang, that is the HTML-artifact pattern: the model emits a self-contained HTML document into a string prop; the app shows a compact status preview inline and opens a full view with Raw and Rendered tabs, rendering the completed document in a sandboxed iframe. Guard it as untrusted input: validate and

normalize it, enforce a Content Security Policy, keep sandbox permissions narrow, and restrict network access. Do not make the artifact the root component - that forces every reply to be a full HTML page.

Where to go from here. You can now trace a message from keyboard to live screen, pick an integration level for your own UI, read the built-in menu, theme it, lock it down, and ship it on your own infrastructure. The natural next steps: scaffold the self-hosted template and trace its files, or pick a domain - support, analytics, travel - and design its menu together. The kitchen is open.

Built from an interactive ten-stop course transcript plus a delivery-considerations Q&A. Diagrams rendered with Mermaid. OpenUI is an open-source project by Thesys - github.com/thesysdev/openui.